

Boosting, AdaBoost

Si con el **bagging** los distintos **estimadores se inducen en paralelo** (o se puede hacer, porque son independientes), con el **boosting se hace secuencialmente**.

Para la inducción de **cada nuevo estimador se tienen en cuenta los resultados de los estimadores anteriores**, para incidir en las observaciones que los anteriores clasificadores no han clasificado correctamente.

De esta manera, **los nuevos estimadores complementan a los anteriores y la calidad del clasificador maestro no depende del azar**.

Es decir, no se necesita un alto número de estimadores para reducir esta dependencia por la ley de los grandes números.

El **nombre de boosting** viene de que con un planteamiento así **se potencia** (boost en inglés) **la calidad de los resultados de clasificadores débiles**.

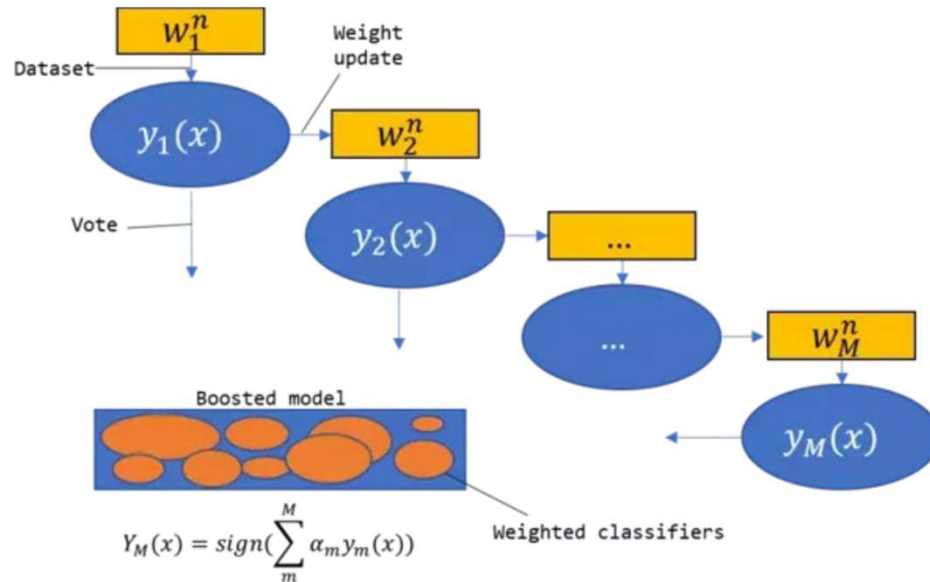
Este planteamiento general **se puede concretar de múltiples maneras**, según como se hagan dos cosas:

1. La **selección de las observaciones** para entrenar cada nuevo estimador, de acuerdo a los resultados del entrenamiento de los anteriores estimadores.
2. La **ponderación de los resultados** de los distintos estimadores.

Con Adaboost se utilizan **estimadores** (árboles de decisión) **muy sencillos** incluso árboles de decisión con únicamente **un nodo raíz y dos nodos hojas**, a los que se conoce como **stumps**.

Pero **el dataset de entrenamiento** utilizado para entrenar cada uno no depende del azar, sino que **se hace de acuerdo al peso que tienen las distintas observaciones**.

El peso de cada observación, es decir, **la probabilidad para seleccionar una observación** para un clasificador **es inicialmente la misma para todos** e igual a **$1/n$** , siendo **n** el **número de observaciones** del dataset de entrenamiento.



Y se revisa tras inducir cada nuevo clasificador, para reducir el del las observaciones correctamente clasificadas.

De esta forma, los sucesivos estimadores **se centran más en las observaciones incorrectamente clasificadas** por los anteriores.

Finalmente, cuando todos los clasificadores individuales se han inducido, **se establece un peso para cada clasificador individual**, de manera que no todos cuentan igual para la clasificación final.

El peso para cada clasificador se establece **mediante un proceso de entrenamiento**. Es decir, también hay que entrenar el clasificador maestro.

La clase que implementa AdaBoost en Scikit-Learn es `AdaBoostClassifier`.

PARÁMETROS DEL CONSTRUCTOR	estimator	Estimador o clasificador de base con el que se construye el conjunto (ensemble). Por defecto es <code>DecisionTreeClassifier(max_depth=1)</code> . Debe tener soporte para ponderación de muestras (<code>sample weithging</code>).
	n_estimators	Máximo número de estimadores con el que se termina el proceso de boosting. El valor por defecto es 50.
	learning_rate	Peso que se aplica a cada clasificador en cada nueva iteración del boosting. Con un valor alto se incrementa su contribución. Con valores mayores para el número de clasificadores (<code>n_estimators</code>), deberían usarse menores valores para <code>learning_rate</code> .
	algorithm	Pueden utilizarse 'SAMME' y 'SAMME.R'. La opción por defecto es 'SAMME.R' porque normalmente converge antes que el primero pero requiere que el estimador de base (dado en el parámetro <code>estimator</code>) debe tener soporte para el cálculo de las probabilidades de las clases. Esto significa, más exactamente, que debe tener un método <code>predict_proba</code>). Este es el caso para la mayoría de clasificadores de Scikit-Learn.
	random_state	Semilla para la generación de números aleatorios que se utiliza para la selección de observaciones de los datasets para entrenamiento de los sucesivos clasificadores. Hay que dar el mismo valor para este parámetro para obtener los mismos resultados en sucesivas ejecuciones del proceso de entrenamiento.

Ejemplo 7 Titanic Adaboost

Partimos de todo el preprocesamiento que se realizó en el ejemplo 5 Titanic RandomForest

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import accuracy_score
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.ensemble import AdaBoostClassifier
```

```
df=pd.read_csv('https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv')

df.drop(['PassengerId', 'Name', 'Ticket', 'Cabin'], axis=1,
inplace=True)

df['Age'] = df['Age'].fillna(df['Age'].mean())
df.dropna(subset=['Embarked'], inplace=True)

for atrib in ['Sex', 'Embarked']:
    df[atrib] = LabelEncoder().fit_transform(df[atrib].values)
target = 'Survived'
df[target]=pd.Categorical(df[target])
x = df.drop(target, axis=1)
y = df[target]

RANDOM_SEED=28
x_train, x_test, y_train, y_test = train_test_split(x, y,
test_size=0.2, random_state=RANDOM_SEED)
```

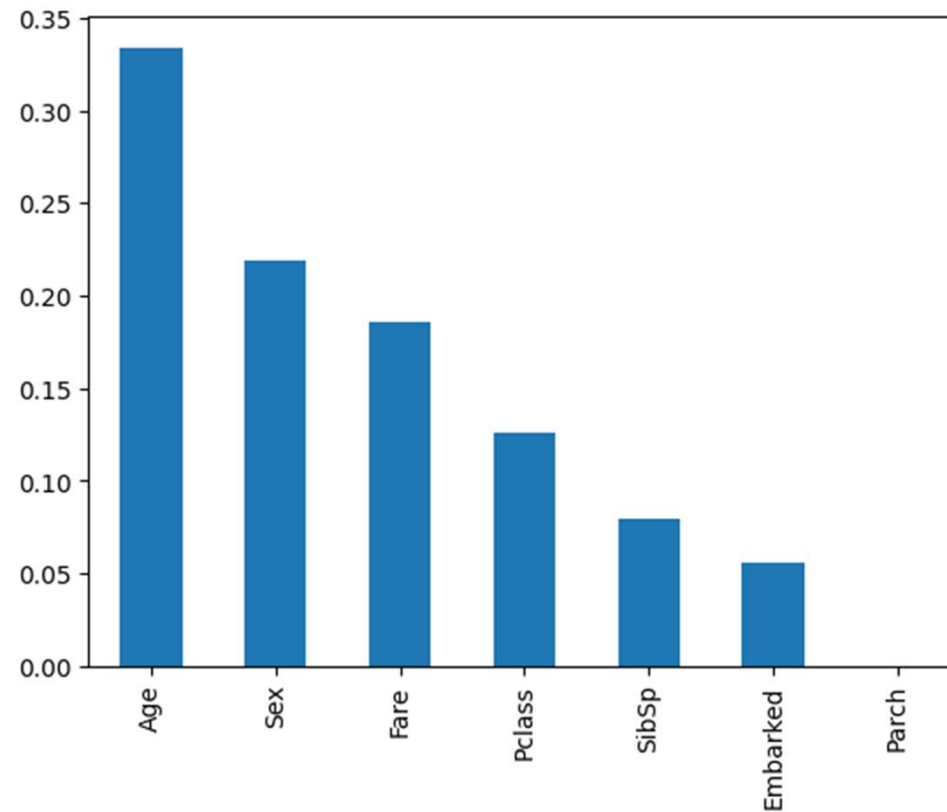
```
# AdaBoost
clf = AdaBoostClassifier(random_state = RANDOM_SEED)
clf.fit(x_train, y_train)

# Validación

y_pred = clf.predict(x_test)
print("accuracy score: ", accuracy_score(y_test, y_pred))
```

accuracy score: 0.8089887640449438

```
pd.Series(clf.feature_importances_,  
index=clf.feature_names_in_).nlargest(clf.feature_names_in_.shape[0]).plot(kind='bar')
```




```
# Ahora con árboles de clasificación (stumps) con max_depth=2

clf = AdaBoostClassifier(random_state = RANDOM_SEED, estimator
= DecisionTreeClassifier(max_depth=2)

clf.fit(x_train, y_train)

y_pred = clf.predict(x_test)

print("accuracy score: ", accuracy_score(y_test, y_pred))
```

accuracy score: 0.8146067415730337

GridSearchCV para intentar encontrar combinación óptima de hiperparámetros.

```
param_dist = {
    'estimator': [DecisionTreeClassifier(random_state=RANDOM_SEED, max_depth=1), DecisionTreeClassifier(random_state=RANDOM_SEED, max_depth=2)],
    # 'estimator': [DecisionTreeClassifier(random_state=RANDOM_SEED)],
    # 'estimator__max_depth': [1, 2],
    'n_estimators': [25, 50, 100, 200], # Valor por defecto: 50.
    'learning_rate': [0.25, 0.5, 1, 2, 4, 8], # Valor por defecto: 1. "trade-off between the learning_rate and n_estimators parameters". Se
entiende que con más clasificadores debería ser menor el valor de learning_rate.
}
```

```
grid_clf = GridSearchCV(estimator = AdaBoostClassifier(),
param_grid = param_dist, cv = 5)
```

```
grid_clf.fit(x, y)
```

```
print(grid_clf.best_params_)
```

```
print(grid_clf.best_score_)
```

```
{'estimator': DecisionTreeClassifier(max_depth=2, random_state=28), 'learning_rate': 1, 'n_estimators': 100}
0.824528661207389
```

```
# Ahora se entrena el mejor clasificador encontrado y se valida  
con validación simple
```

```
clf = grid_clf.best_estimator_
```

```
clf.fit(x_train, y_train)
```

```
# Validación
```

```
y_pred = clf.predict(x_test)
```

```
print("accuracy score: ", accuracy_score(y_test, y_pred))
```

accuracy score: 0.8370786516853933

```
pd.Series(clf.feature_importances_,  
index=clf.feature_names_in_).nlargest(clf.feature_names_in_.shape[0]).plot(kind='bar')
```

